

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

Nishant Makwana(1BF24CS202)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

February to June 2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “**OPERATING SYSTEMS**” carried out by Nishant Makwana(1BF24CS202), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Operating Systems Lab - **(23CS4PCOPS)** work prescribed for the said degree.

Prof. Lakshmi Neelima
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	5-18
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	19-23
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	24-27
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	28-32
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	33-38
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	39-42
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	43-48

Course Outcomes:

CO1	Apply the different concepts and functionalities of Operating Systems.
CO2	Analyse various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating Systems.

CO4	Conduct practical experiments to implement the functionalities of Operating systems.
------------	--

Lab Program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS:

```
#include <stdio.h>
int main() {
    int n, i;    int at[20], bt[20], ct[20], tat[20],
    wt[20], rt[20];    float avg_wt = 0, avg_tat = 0,
    avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0;i<n;i++){

        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);
    }
    ct[0] = at[0] + bt[0];

    for(i=1;i<n;i++){

        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];    else
            ct[i] = ct[i-1] + bt[i];
    }

    for(i=0;i<n;i++){
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];    rt[i]
        = wt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
        avg_rt += rt[i];
    }
}
```

```

    }
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(i=0;i<n;i++){
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
printf("\nAverage Response Time = %.2f\n", avg_rt/n);

    return 0; }

```

Output:

```

Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 5

Process AT      BT      CT      TAT      WT      RT
P1      0      5      5      5      0      0
P2      1      3      8      7      4      4
P3      2      5     13     11      6      6

Average Waiting Time = 3.33
Average Turnaround Time = 7.67
Average Response Time = 3.33

```

b) SJF(Preemptive):

```
#include <stdio.h>
#include <limits.h>

int main(){

    int n,i,time=0,completed=0;
    int at[20], bt[20], rem_bt[20];
    int ct[20], tat[20], wt[20], rt[20];
    int first[20]={0};

    float avg_wt=0, avg_tat=0, avg_rt=0;

    printf("Enter number of processes: ");

    scanf("%d",&n);
    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0;i<n;i++){

        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);

        rem_bt[i]=bt[i];
        rt[i]=-1;
    }

    while(completed<n){

        int shortest=-1;    int
        min_bt=INT_MAX;

        for(i=0;i<n;i++){    if(at[i]<=time && rem_bt[i]>0
        && rem_bt[i]<min_bt){    min_bt=rem_bt[i];
            shortest=i;
        }
    }

    if(shortest==-1){
        time++;    continue;
```

```

    }

    if(rt[shortest]==-1)
rt[shortest]=time-at[shortest];

    rem_bt[shortest]--;
time++;

    if(rem_bt[shortest]==0){
completed++;
ct[shortest]=time;

    tat[shortest]=ct[shortest]-at[shortest];
wt[shortest]=tat[shortest]-bt[shortest];

    avg_wt+=wt[shortest];
avg_tat+=tat[shortest];
avg_rt+=rt[shortest];
    }
}

printf("\nProcess AT BT CT TAT WT RT\n");
for(i=0;i<n;i++){    printf("P%d    %d %d
%d %d %d %d\n",
i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

Output:

```

Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4

Process AT BT CT TAT WT RT
P1      0  5  8  8   3  0
P2      1  3  4  3   0  0
P3      2  4 12 10   6  6

Average Waiting Time = 3.00
Average Turnaround Time = 7.00
Average Response Time = 2.00

```


SJF (Non-Preemptive):

```
#include <stdio.h>
#include <limits.h>

int main() { int n, i, time = 0, completed = 0;
int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
int done[20] = {0};
float avg_wt = 0, avg_tat = 0, avg_rt = 0;

printf("Enter number of processes: ");

scanf("%d",&n);
printf("Enter Arrival Time and Burst Time:\n");

for(i=0;i<n;i++){

printf("P%d AT: ",i+1);
scanf("%d",&at[i]);

printf("P%d BT: ",i+1);
scanf("%d",&bt[i]);
}

while(completed < n){

int shortest = -1;    int
min_bt = INT_MAX;

for(i=0;i<n;i++){    if(at[i] <= time &&
done[i]==0 && bt[i] < min_bt){    min_bt = bt[i];
shortest = i;
}
}

if(shortest == -1){
time++;    continue;
}

rt[shortest] = time - at[shortest];

time += bt[shortest];

ct[shortest] = time;
```

```

    tat[shortest] = ct[shortest] - at[shortest];
    wt[shortest] = tat[shortest] - bt[shortest];

    done[shortest] = 1;
    completed++;

    avg_wt += wt[shortest];
    avg_tat += tat[shortest];
    avg_rt
    += rt[shortest];
}
printf("\nProcess AT BT CT TAT WT RT\n");
for(i=0;i<n;i++){    printf("P%d    %d %d
%d %d %d %d\n",
i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0; }

```

Output:

```

Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4

Process AT BT CT TAT WT RT
P1      0  5  5  5   0  0
P2      1  3  8  7   4  4
P3      2  4 12 10   6  6

Average Waiting Time = 3.33
Average Turnaround Time = 7.33
Average Response Time = 3.33

```

c) Priority(Preemptive):

```
#include <stdio.h>
#include <limits.h>

int main() {    int
n,i,time=0,completed=0;    int
at[20],bt[20],pr[20],rem_bt[20];    int
ct[20],tat[20],wt[20],rt[20];
    float avg_wt=0,avg_tat=0,avg_rt=0;
    printf("Enter number of processes: ");
scanf("%d",&n);
    printf("Enter Arrival Time, Burst Time and Priority:\n");

    for(i=0;i<n;i++)

    {
        printf("P%d AT: ",i+1);
scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
scanf("%d",&bt[i]);

        printf("P%d Priority: ",i+1);
scanf("%d",&pr[i]);

        rem_bt[i]=bt[i];
rt[i]=-1;
    }

    while(completed<n)
    {        int highest=-1;
int min_pr=INT_MAX;

        for(i=0;i<n;i++)
        {
            if(at[i]<=time && rem_bt[i]>0 && pr[i]<min_pr)
            {
min_pr=pr[i];
highest=i;
```

```

    }
}

if(highest==-1)
{
time++;
continue;
}

if(rt[highest]==-1)
rt[highest] = time - at[highest];

rem_bt[highest]--;
time++;
if(rem_bt[highest]==0)
{
    completed++;

    ct[highest]=time;
    tat[highest]=ct[highest]-at[highest];
    wt[highest]=tat[highest]-bt[highest];

    avg_wt+=wt[highest];
    avg_tat+=tat[highest];
    avg_rt+=rt[highest];
}
}
printf("\nProcess AT BT PR CT TAT WT RT\n");

for(i=0;i<n;i++)

{
    printf("P%d    %d %d %d %d %d %d %d\n",
i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0; }

```

Output:

```

Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority:
P1 AT: 0
P1 BT: 5
P1 Priority: 3
P2 AT: 1
P2 BT: 3
P2 Priority: 1
P3 AT: 3
P3 BT: 4
P3 Priority: 2

Process AT BT PR CT TAT WT RT
P1      0  5  3 12 12  7  0
P2      1  3  1  4  3  0  0
P3      3  4  2  8  5  1  1

Average Waiting Time = 2.67
Average Turnaround Time = 6.67
Average Response Time = 0.33

```

Priority (Non-preemptive):

```

#include <stdio.h>
#include <limits.h>

int main() {    int
n,i,time=0,completed=0;    int
at[20],bt[20],pr[20];    int
ct[20],tat[20],wt[20],rt[20];
int done[20]={0};

    float avg_wt=0,avg_tat=0,avg_rt=0;

    printf("Enter number of processes: ");

    scanf("%d",&n);
    printf("Enter Arrival Time, Burst Time and Priority:\n");
    for(i=0;i<n;i++)
    {
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);

        printf("P%d Priority: ",i+1);
        scanf("%d",&pr[i]);
    }

    while(completed<n)

```

```

    {        int highest=-1;
int min_pr=INT_MAX;

    for(i=0;i<n;i++)
    {
        if(at[i]<=time && done[i]==0 && pr[i]<min_pr)
        {
min_pr=pr[i];
highest=i;
        }
    }

    if(highest==-1)
    {
time++;
continue;
    }
    rt[highest] = time - at[highest];

    time += bt[highest];

ct[highest] = time;

    tat[highest] = ct[highest] - at[highest];
wt[highest] = tat[highest] - bt[highest];

    avg_wt += wt[highest];
avg_tat += tat[highest];
avg_rt
+= rt[highest];

    done[highest]=1;
completed++;
    }
    printf("\nProcess AT BT PR CT TAT WT RT\n");
    for(i=0;i<n;i++)
    {
        printf("P%d    %d %d %d %d %d %d %d\n",
i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
    }

```

```
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Response Time = %.2f\n",avg_rt/n);

    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority:
P1 AT: 0
P1 BT: 5
P1 Priority: 3
P2 AT: 1
P2 BT: 3
P2 Priority: 1
P3 AT: 2
P3 BT: 3
P3 Priority: 2

Process AT BT PR CT TAT WT RT
P1      0  5  3  5  5  0  0
P2      1  3  1  8  7  4  4
P3      2  3  2  11  9  6  6

Average Waiting Time = 3.33
Average Turnaround Time = 7.00
Average Response Time = 3.33
```

d) Round Robin:

```
#include <stdio.h>
```

```
void roundRobin(int n, int at[], int bt[], int tq) {  
    int rt[n], ct[n], wt[n], tat[n], resp[n];
```

```
    for (int i = 0; i < n; i++) {  
        rt[i] = bt[i];    resp[i] = -  
        1;  
    }  
    int time = 0, completed = 0;  
    while (completed < n) {  
        int idle = 1;
```

```
        for (int i = 0; i < n; i++) {  
            if (at[i] <= time && rt[i] > 0) {  
                idle = 0;
```

```
                if (resp[i] == -1) {  
                    resp[i] = time - at[i];  
                }    if  
                (rt[i] > tq) {  
                    time += tq;  
                    rt[i] -= tq;    } else  
                {  
                    time +=  
                    rt[i];    ct[i] =  
                    time;    rt[i] =  
                    0;  
                    completed++;  
                }  
            }  
        }    if
```

```
        (idle) {  
            time++;  
        }  
    }
```

```
    float total_wt = 0, total_tat = 0, total_rt = 0;
```



```

    for (int i = 0; i < n; i++) {
tat[i] = ct[i] - at[i];    wt[i]
= tat[i] - bt[i];

        total_wt += wt[i];
total_tat += tat[i];    total_rt
+= resp[i];
    }

    printf("\nTime Quantum = %d\n", tq);
printf("Process\tAT\tBT\tCT\tWT\tTAT\tRT\n");

    for (int i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], wt[i], tat[i], resp[i]);
    }

    printf("Average WT = %.2f\n", total_wt / n);
printf("Average TAT = %.2f\n", total_tat / n);    printf("Average
RT = %.2f\n", total_rt / n);
}

int main() {
int n;

    printf("Enter number of processes: ");
scanf("%d", &n);

    int at[n], bt[n];

    for (int i = 0; i < n; i++) {
printf("\nProcess P%d\n", i + 1);

        printf("Arrival Time: ");
scanf("%d", &at[i]);

        printf("Burst Time: ");
scanf("%d", &bt[i]);
    }

    int num_q, tq;

```

```

    printf("\nEnter number of different Time Quantum: ");
    scanf("%d", &num_q);

    for (int i = 0; i < num_q; i++) {
        printf("\nEnter Time Quantum %d: ", i + 1);
        scanf("%d", &tq);

        roundRobin(n, at, bt, tq);
    }

    return 0;
}

```

Output:

```

Enter number of processes: 3

Process P1
Arrival Time: 1
Burst Time: 3

Process P2
Arrival Time: 1
Burst Time: 4

Process P3
Arrival Time: 1
Burst Time: 5

Enter number of different Time Quantum: 2

Enter Time Quantum 1: 3

Time Quantum = 3
Process AT      BT      CT      WT      TAT      RT
P1      1        3        4        0        3        0
P2      1        4       11        6       10        3
P3      1        5       13        7       12        6
Average WT = 4.33
Average TAT = 8.33
Average RT = 3.00

Enter Time Quantum 2: 5

Time Quantum = 5
Process AT      BT      CT      WT      TAT      RT
P1      1        3        4        0        3        0
P2      1        4        8        3        7        3
P3      1        5       13        7       12        7
Average WT = 3.33
Average TAT = 7.33
Average RT = 3.33

```

Lab Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <string.h>
#define MAX_PROCESSES 100

typedef struct {    int
process_id;    int
arrival_time;    int
burst_time;    int
remaining_time;    int
type;    int
completion_time;
int waiting_time;    int
turnaround_time;    int
started;    int
start_time;
} Process;

typedef struct {    int
items[MAX_PROCESSES];
    int front, rear;
} Queue;

void initQueue(Queue* q) {
q->front = 0;    q->rear = -
1;
}

int isEmpty(Queue* q) {
return q->rear < q->front;
}

void enqueue(Queue* q, int value) {    if
(q->rear < MAX_PROCESSES - 1) {
q->rear++;    q->items[q->rear] =
value;
    }
}
```

```

int dequeue(Queue* q) {    if
(!isEmpty(q)) {          int val = q-
>items[q->front];        q-
>front++;                return val;
    }
return -1;
}

```

```

int compareArrival(const void* a, const void* b) {
    Process* p1 = (Process*)a;    Process* p2 =
(Process*)b;    if (p1->arrival_time < p2-
>arrival_time)    return -1;    else if (p1-
>arrival_time > p2->arrival_time)    return
1;    else    return 0;
}

```

```

int main() {
int n;
    Process processes[MAX_PROCESSES];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
printf("\nProcess %d\n", i);    printf("Enter
arrival time: ");    scanf("%d",
&processes[i].arrival_time);    printf("Enter
burst time: ");    scanf("%d",
&processes[i].burst_time);    printf("Enter
type (0 = System, 1 = User): ");    scanf("%d",
&processes[i].type);

        processes[i].process_id = i;
processes[i].remaining_time = processes[i].burst_time;
processes[i].completion_time = 0;
processes[i].waiting_time = 0;
processes[i].turnaround_time = 0;    processes[i].started =
0;    processes[i].start_time = -1;
    }

    qsort(processes, n, sizeof(Process), compareArrival);
}

```

```

Queue systemQueue, userQueue;
initQueue(&systemQueue);
initQueue(&userQueue);

```

```

int completed = 0;
int current_time = 0;
int i = 0;
Process* current_process = NULL;

```

```

while (completed < n) {      while (i < n &&
processes[i].arrival_time <= current_time) {      if
(processes[i].type == 0)      enqueue(&systemQueue, i);
      else
      enqueue(&userQueue, i);
i++;
}

```

```

if (current_process != NULL) {      if (current_process->type
== 1 && !isEmpty(&systemQueue)) {      enqueue(&userQueue,
current_process - processes);      current_process = NULL;
      }
}

```

```

if (current_process == NULL) {
if (!isEmpty(&systemQueue)) {      int
idx = dequeue(&systemQueue);
current_process = &processes[idx];      }
else if (!isEmpty(&userQueue)) {
int idx = dequeue(&userQueue);
current_process = &processes[idx];
      } else {
current_time++;
continue;
      }
}

```

```

if (current_process->started == 0) {      current_process-
>start_time = current_time;      current_process->started = 1;
}

```

```

current_process->remaining_time--;
current_time++;

```

```

        if (current_process->remaining_time == 0) {
            current_process->completion_time =
current_time;
            current_process->turnaround_time = current_process->completion_time -
current_process-
>arrival_time;
            current_process->waiting_time = current_process->turnaround_time -
current_process-
>burst_time;
            completed++;
            current_process = NULL;
        }
    }

    printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
    double total_wt = 0, total_tat = 0;
    for (int j = 0; j < n; j++) {
        printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
processes[j].process_id,
            (processes[j].type == 0) ? "System" : "User",
processes[j].arrival_time,
            processes[j].burst_time,
processes[j].completion_time,
processes[j].waiting_time,
processes[j].turnaround_time);

        total_wt += processes[j].waiting_time;
        total_tat += processes[j].turnaround_time;
    }

    printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
    printf("Average Turnaround Time: %.2f\n", total_tat / n);

    return 0;
}

```

Output:

Enter number of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = System, 1 = User): 0

Process 3

Enter arrival time: 3

Enter burst time: 2

Enter type (0 = System, 1 = User): 1

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
2	System	1	4	9	4	8
1	User	2	3	12	7	10
3	User	3	2	14	9	11

Average Waiting Time: 5.00

Average Turnaround Time: 8.50

Lab Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

```
#include <stdio.h>
#include <math.h>

#define MAX 10

typedef struct {
    int id;    int
    burst;    int
    deadline;    int
    period;    int
    weight;    int
    ct, wt, tat;
} Process;

void edf(Process p[], int n) {
    float util = 0;

    for (int i = 0; i < n; i++)    util +=
(float)p[i].burst / p[i].deadline;

    for (int i = 0; i < n - 1; i++)    for (int j
= i + 1; j < n; j++)    if (p[i].deadline >
p[j].deadline) {    Process t = p[i];
p[i] = p[j]; p[j] = t;
    }

    int time = 0;    for (int i = 0; i
< n; i++) {    time +=
p[i].burst;    p[i].ct = time;
p[i].tat = p[i].ct;    p[i].wt =
p[i].tat - p[i].burst;
    }

    printf("\n===== EDF Scheduling =====\n");
    printf("Utilization = %.2f\n", util);    printf("Schedulable
if U <= 1\n");    printf("ID\tBT\tDL\tCT\tWT\tTAT\n");
    for (int i = 0; i < n; i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",    p[i].id,
p[i].burst, p[i].deadline,    p[i].ct, p[i].wt, p[i].tat);
```



```

}

void rms(Process p[], int n) {
float util = 0;

    for (int i = 0; i < n; i++)        util +=
(float)p[i].burst / p[i].period;
    float bound = n * (pow(2, (float)1/n) - 1);

    for (int i = 0; i < n - 1; i++)    for (int j

= i + 1; j < n; j++)                if (p[i].period >
p[j].period) {                      Process t = p[i]; p[i]
= p[j]; p[j] = t;
    }

    int time = 0;    for (int i = 0; i
< n; i++) {        time +=
p[i].burst;        p[i].ct = time;
p[i].tat = p[i].ct;    p[i].wt =
p[i].tat - p[i].burst;
    }

    printf("\n===== RMS Scheduling =====\n");
    printf("Utilization = %.2f\n", util); printf("RM
Bound = %.4f\n", bound);

    if (util <= bound)
printf("Schedulable\n");    else
printf("Not guaranteed schedulable\n");

    printf("ID\tBT\tPeriod\tCT\tWT\tTAT\n")
; for (int i = 0; i < n; i++)
printf("%d\t%d\t%d\t%d\t%d\t%d\n",
p[i].id, p[i].burst, p[i].period,
p[i].ct, p[i].wt, p[i].tat);
}

void proportional(Process p[], int n) {
int total_weight = 0;

    for (int i = 0; i < n; i++)
total_weight += p[i].weight;

```

```
printf("\n===== Proportional Scheduling =====\n");
printf("Total Weight = %d\n", total_weight);
```

```
printf("ID\tWeight\tCPU Share\n");
```

```
for (int i = 0; i < n; i++) {    float share =
(float)p[i].weight / total_weight;
printf("%d\t%d\t%.2f\n", p[i].id, p[i].weight, share);
}
}
```

```
int main() {
int n;
Process p1[MAX], p2[MAX], p3[MAX];
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
```

```
for (int i = 0; i < n; i++) {
printf("\nProcess %d\n", i);
p1[i].id = p2[i].id = p3[i].id = i;
```

```
printf("Burst Time: ");
```

```
scanf("%d", &p1[i].burst);
```

```
printf("Deadline (EDF): ");
scanf("%d", &p1[i].deadline);
```

```
printf("Period (RMS): ");
scanf("%d", &p1[i].period);
printf("Weight (Proportional): ");
scanf("%d", &p1[i].weight);
```

```
p2[i] = p1[i];
p3[i] = p1[i];
}
```

```
edf(p1, n);
rms(p2, n);
proportional(p3, n);
```

```
return 0;
```

}

Output:

```
Enter number of processes: 3

Process 0
Burst Time: 3
Deadline (EDF): 7
Period (RMS): 20
Weight (Proportional): 5

Process 1
Burst Time: 2
Deadline (EDF): 4
Period (RMS): 5
Weight (Proportional): 7

Process 2
Burst Time: 2
Deadline (EDF): 8
Period (RMS): 10
Weight (Proportional): 4

===== EDF Scheduling =====
Utilization = 1.18
Schedulable if  $U \leq 1$ 

| ID | BT | DL | CT | WT | TAT |
|----|----|----|----|----|-----|
| 1  | 2  | 4  | 2  | 0  | 2   |
| 0  | 3  | 7  | 5  | 2  | 5   |
| 2  | 2  | 8  | 7  | 5  | 7   |


```

```
===== RMS Scheduling =====
Utilization = 0.75
RM Bound = 0.7798
Schedulable


| ID | BT | Period | CT | WT | TAT |
|----|----|--------|----|----|-----|
| 1  | 2  | 5      | 2  | 0  | 2   |
| 2  | 2  | 10     | 4  | 2  | 4   |
| 0  | 3  | 20     | 7  | 4  | 7   |



===== Proportional Scheduling =====
Total Weight = 16


| ID | Weight | CPU Share |
|----|--------|-----------|
| 0  | 5      | 0.31      |
| 1  | 7      | 0.44      |
| 2  | 4      | 0.25      |


```

Lab Program 4:

Write a C program to simulate:

a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem

a) Producer-Consumer problem using semaphores

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 15

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full; pthread_mutex_t
mutex;

void* producer(void* arg) {    for
(int i = 0; i < ITEMS; i++) {

    sem_wait(&empty);
    pthread_mutex_lock(&mutex);

    buffer[in] = i;    printf("Produced: %d at
buffer[%d]\n", i, in);

    in = (in + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&full);

    usleep(100000);
}
return NULL;
}

void* consumer(void* arg) {
for (int i = 0; i < ITEMS; i++) {
```

```

    sem_wait(&full);
    pthread_mutex_lock(&mutex);

    int item = buffer[out];    printf("Consumed: %d
from buffer[%d]\n", item, out);

    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);

    usleep(150000);
}
return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Output:

```
"C:\Users\BMSCE\Desktop\ve" X + v
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Consumed: 4 from buffer[4]
Consumed: 5 from buffer[0]
Consumed: 6 from buffer[1]

Process returned 0 (0x0)   execution time : 14.072 s
Press any key to continue.
|
```

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5 // fewer philosophers

sem_t forks[N];

void* philosopher(void* num) {
    int id = *(int*)num;

    printf("Philosopher %d is thinking.\n", id);

    // Deadlock prevention: last philosopher picks right first    if (id == N -
1) {        sem_wait(&forks[(id + 1) % N]); // right fork
    printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);

        sem_wait(&forks[id]); // left fork
    printf("Philosopher %d picked up left fork %d.\n", id, id);    }
```

```

else {
    sem_wait(&forks[id]); // left fork
    printf("Philosopher %d picked up left fork %d.\n", id,
id);

    sem_wait(&forks[(id + 1) % N]); // right fork
    printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) %
N);
}
    printf("Philosopher %d is eating.\n", id);
    sem_post(&forks[id]);
    sem_post(&forks[(id + 1) % N]);

    printf("Philosopher %d put down forks %d and %d.\n",
id, id, (id + 1) % N);

    return NULL;
}

int main() {
    pthread_t phil[N];
    int ids[N];

    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }

    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(phil[i], NULL);
    }

    return 0;
}

```

Output:

Philosopher 0 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 1 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 1 picked up right fork 2.
Philosopher 1 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 3 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 picked up right fork 4.
Philosopher 3 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 2 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 4 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 picked up left fork 4.
Philosopher 4 is eating.
Philosopher 4 put down forks 4 and 0.

Lab Program 5:

Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection

a) Bankers' algorithm for the purpose of deadlock avoidance

```
#include <stdio.h>

int main() {    int n,
m, i, j, k, ch;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    printf("Enter no. of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], work[m], finish[n], safe[n];

    printf("Enter          Allocation
Matrix:\n");          for(i=0;i<n;i++)
for(j=0;j<m;j++)
scanf("%d",&alloc[i][j]);

    printf("Enter Max
Matrix:\n"); for(i=0;i<n;i++)
for(j=0;j<m;j++)
scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");
    for(i=0;i<m;i++)
    scanf("%d",&avail[i]);

    for(i=0;i<n;i++)
    for(j=0;j<m;j++)
    need[i][j]=max[i][j]-alloc[i][j];

    printf("\n1. Safety Algorithm");
    printf("\n2. Resource Request");
    printf("\nEnter choice(1 or 2): ");
    scanf("%d",&ch);
```

```

switch(ch) {

    case 1: {      for(i=0;i<m;i++)
work[i]=avail[i];
for(i=0;i<n;i++) finish[i]=0;
                int count=0;
while(count<n) {      int found=0;
for(i=0;i<n;i++) {
if(finish[i]==0) {
for(j=0;j<m;j++)
if(need[i][j]>work[j])
break;          if(j==m) {
for(k=0;k<m;k++)
work[k]+=alloc[i][k];
safe[count++]=i;
finish[i]=1;          found=1;
                }
            }
        }
        if(found==0)
break;
    }
    if(count==n) {
printf("\nSAFE STATE\nSafe Sequence:
");
for(i=0;i<n;i++)          printf("P%d ",safe[i]);
    }
    else
        printf("\nUNSAFE STATE");
    break;
}

    case 2: {      int p, req[m];
printf("Enter process number: ");
scanf("%d",&p);
printf("Enter Request Vector:\n");
for(i=0;i<m;i++)
for(i=0;i<m;i++) {
if(req[i]>need[p][i]) {
printf("Error: Exceeded Need");
return 0;          }
if(req[i]>avail[i]) {
printf("Resources not available");
return 0;
        }
    }
}

```

```

        for(i=0;i<m;i++) {          avail[i]-
=req[i];          alloc[p][i]+=req[i];
need[p][i]-=req[i];
        }
        printf("\nRequest Granted\n");
break; } default:
printf("Invalid Choice");
        }
return 0;
}

```

Output:

```

Enter no. of processes: 5
Enter no. of resources: 3
Enter Allocation Matrix:
0
1
0
2
0
0
0
3
0
2
2
1
1
0
0
2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

1. Safety Algorithm
2. Resource Request
Enter choice(1 or 2): 1

SAFE STATE
Safe Sequence: P1 P3 P4 P0 P2

```

```

Enter no. of processes: 5
Enter no. of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

1. Safety Algorithm
2. Resource Request
Enter choice(1 or 2): 2
Enter process number: 1
Enter Request Vector:
Error: Exceeded Need
Process returned 0 (0x0)   execution time : 47.191 s
Press any key to continue.

```

b) Deadlock Detection

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
    scanf("%d", &m);
```

```
    int Allocation[n][m],  
    Request[n][m]; int Available[m];
```

```
    printf("\nEnter Allocation  
Matrix:\n"); for(i = 0; i < n; i++) { for(j  
= 0; j < m; j++) { scanf("%d",  
&Allocation[i][j]);  
        }  
    }
```

```
    printf("\nEnter Request Matrix:\n");  
    for(i = 0; i < n; i++) {        for(j = 0; j  
< m; j++) {            scanf("%d",  
&Request[i][j]);  
        }  
    }
```

```
    printf("\nEnter Available  
Resources:\n"); for(i = 0; i < m; i++) {  
    scanf("%d", &Available[i]);  
    }  
    int Work[m], Finish[n];  
    for(i = 0; i < m; i++)  
    Work[i] = Available[i];
```

```
    for(i = 0; i < n; i++)  
        Finish[i] = 0;
```

```
    int count = 0;
```

```
    while(count < n) {
```

```
        int found = 0;
```

```
        for(i = 0; i < n; i++) {
```

```
            if(Finish[i] == 0) {
```

```
                for(j = 0; j < m; j++) {  
                    if(Request[i][j] > Work[j])  
                        break;
```

```

        }
        if(j == m)
{
        for(k = 0; k < m; k++) {
Work[k] += Allocation[i][k];
        }

Finish[i] = 1;
found = 1;

count++;
    }
}
}

    if(found == 0)
break;
    }

    int deadlock = 0;

    for(i = 0; i < n; i++) {        if(Finish[i] == 0) {
deadlock = 1;        printf("Process P%d is
deadlocked.\n", i);
    }
}

    if(deadlock == 0)        printf("\nNo
Deadlock Detected.\n");

    return 0; }

```

Output:

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:

0	1	0
2	0	0
3	0	3
2	1	1
0	0	2

Enter Request Matrix:

0	0	0
2	0	2
0	0	0
1	0	0
0	0	2

Enter Available Resources:

0	0	0
---	---	---

No Deadlock Detected.

Lab Program 6:

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>

int main() {
    int nb, np, i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    int blockSize[nb];

    printf("Enter sizes of %d memory blocks:\n", nb);
    for(i = 0; i < nb; i++) {        scanf("%d",
    &blockSize[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &np);

    int process[np];

    printf("Enter sizes of %d processes:\n",
    np);    for(i = 0; i < np; i++) {
    scanf("%d", &process[i]);
    }
    int firstFit[np], bestFit[np], worstFit[np];

    for(i = 0; i < np; i++) {

    firstFit[i] = -1;
    bestFit[i] = -1;
    worstFit[i] = -1;
    }

    int used1[nb];

    for(i = 0; i < nb; i++)
    used1[i] = 0;
```

```

    for(i = 0; i < np; i++) {        for(j = 0; j < nb; j++)
    {            if(!used1[j] && blockSize[j]
    >= process[i]) {
        firstFit[i] = j;
    used1[j] = 1;
    break;
    }
    }
}

```

```

int used2[nb];

```

```

    for(i = 0; i < nb; i++)
used2[i] = 0;

```

```

    for(i = 0; i < np; i++) {
int best = -1;

```

```

        for(j = 0; j < nb; j++) {            if(!used2[j] &&
blockSize[j] >= process[i]) {

```

```

                                if(best == -1 || blockSize[j] < blockSize[best]) {
best = j;
                                }

```

```

        }
    }
}

```

```

    if(best != -1) {
bestFit[i] = best;
used2[best] = 1;
    }
}

```

```

int used3[nb];

```

```

    for(i = 0; i < nb; i++)
used3[i] = 0;

```

```

    for(i = 0; i < np; i++) {
int worst = -1;

```

```

        for(j = 0; j < nb; j++) {            if(!used3[j] &&
blockSize[j] >= process[i]) {

```



```

        if(worst == -1 || blockSize[j] > blockSize[worst]) {
worst = j;
        }
    }

    if(worst != -1) {
worstFit[i] = worst;
used3[worst] = 1;
    }
}

printf("\n--- First Fit ---\n");    printf("Process
No.\tProcess Size\tBlock No.\n");

for(i = 0; i < np; i++) {    if(firstFit[i] != -1)
printf("%d\t%d\t%d\n", i + 1, process[i], firstFit[i] + 1);
else    printf("%d\t%d\tNot Allocated\n", i + 1,
process[i]);
}

printf("\n--- Best Fit ---\n");    printf("Process
No.\tProcess Size\tBlock No.\n");

for(i = 0; i < np; i++) {    if(bestFit[i] != -1)
printf("%d\t%d\t%d\n", i + 1, process[i], bestFit[i] + 1);
else    printf("%d\t%d\tNot Allocated\n", i + 1,
process[i]);
}

printf("\n--- Worst Fit ---\n");    printf("Process
No.\tProcess Size\tBlock No.\n");

for(i = 0; i < np; i++) {    if(worstFit[i] != -1)
printf("%d\t%d\t%d\n", i + 1, process[i], worstFit[i] + 1);
else
    printf("%d\t%d\tNot Allocated\n", i + 1, process[i]);
}

return 0; }

```

Output:

Enter number of memory blocks: 5

Enter sizes of 5 memory blocks:

100 500 200 300 600

Enter number of processes: 4

Enter sizes of 4 processes:

212 417 112 426

--- First Fit ---

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

--- Best Fit ---

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	4
4	426	Not Allocated

Lab Program 7:

Write a C program to simulate page replacement algorithms a) FIFO b) LRU c) Optimal

```
#include <stdio.h>

void fifo(int page[], int n, int f)
{
    int frame[f];
    int i,j,pos=0,faults=0,found;

    for(i=0;i<f;i++)
        frame[i]=-1;
    printf("\nPage\tFrames\t\tStatus\n");

    for(i=0;i<n;i++)
    {
        found=0;

        for(j=0;j<f;j++)
        {
            if(frame[j]==page[i])
            {
                found=1;
                break;
            }
        }

        if(found==0)
        {
            frame[pos]=page[i];
            pos=(pos+1)%f;
            faults++;
        }
        printf("%d\t",page[i]);

        for(j=0;j<f;j++)

        printf("%d ",frame[j]);
        printf("\t%s\n",found?"Hit":"Fault");
    }
}
```

```
    printf("\nTotal Page Faults = %d\n",faults);  
}
```

```
void lru(int page[], int n, int f)  
{    int frame[f],recent[f];  
int i,j,found,faults=0,pos,min;
```

```
    for(i=0;i<f;i++)  
    {  
frame[i]=-1;  
recent[i]=-1;  
    }  
    printf("\nPage\tFrames\t\tStatus\n");
```

```
    for(i=0;i<n;i++)
```

```
    {  
found=0;
```

```
        for(j=0;j<f;j++)  
        {  
            if(frame[j]==page[i])  
            {  
found=1;  
recent[j]=i;  
break;  
            }  
        }
```

```
        if(found==0)  
        {  
faults++;
```

```
        pos=0;  
min=recent[0];
```

```
        for(j=0;j<f;j++)  
        {  
            if(frame[j]==-1)  
            {  
pos=j;  
break;  
            }  
        }
```

```

        if(recent[j]<min)
        {
min=recent[j];
pos=j;
        }
    }

    frame[pos]=page[i];
recent[pos]=i;
    }
    printf("%d\t",page[i]);

    for(j=0;j<f;j++)

printf("%d ",frame[j]);

    printf("\t%s\n",found?"Hit":"Fault");
}

printf("\nTotal Page Faults = %d\n",faults);
}

void optimal(int page[], int n, int f)
{
    int frame[f];
    int i,j,k,found,faults=0,pos,farthest,index;
;

    for(i=0;i<f;i++)
frame[i]=-1;
    printf("\nPage\tFrames\t\tStatus\n");

    for(i=0;i<n;i++)

    {
found=0;
for(j=0;j<f;j
++) {
if(frame[j]=
=page[i])
    {
        found=1;
        break;
    }
}

```

```

    }

    if(found==0)
    {
faults++;
        pos=-1;
        for(j=0;j<f;j++)
        {
if(frame[j]==-1)
            {
pos=j;
break;
            }
        }

        if(pos==-1)
        {
farthest=-1;
pos=0;

            for(j=0;j<f;j++)
            {
index=9999;

                for(k=i+1;k<n;k++)
                {
if(frame[j]==page[k])
                    {
index=k;
break;
                    }
                }
                if(index>farthest)
                {
farthest=index;
pos=j;
                }
            }
        }

        frame[pos]=page[i];

```

```

    }
    printf("%d\t",page[i]);
    for(j=0;j<f;j++)
printf("%d ",frame[j]);

    printf("\t%s\n",found?"Hit":"Fault");
}

printf("\nTotal Page Faults = %d\n",faults);
}

```

```

int main() {
int n,f,i,ch;

printf("Enter Number of Pages: ");
scanf("%d",&n);

int page[n];
printf("Enter Reference String:\n");
for(i=0;i<n;i++)
scanf("%d",&page[i]);

printf("Enter Number of Frames: ");
scanf("%d",&f);

printf("\n1. FIFO");
printf("\n2. LRU");
printf("\n3. Optimal");
printf("\nEnter Choice: ");
scanf("%d",&ch);

```

```

switch(ch)
{
case 1:
fifo(page,n,f);
break;
case 2:
lru(page,n,f);
break;

```

```

        case 3:
optimal(page,n,f);
break;

default:
    printf("Invalid Choice");
    }

    return 0;
}

```

Output:

```

Enter Number of Pages: 7
Enter Reference String:
1 2 3 4 5 6 7
Enter Number of Frames: 4

1. FIFO
2. LRU
3. Optimal
Enter Choice: 1

Page      Frames      Status
1         1 -1 -1 -1      Fault
2         1 2 -1 -1      Fault
3         1 2 3 -1      Fault
4         1 2 3 4      Fault
5         5 2 3 4      Fault
6         5 6 3 4      Fault
7         5 6 7 4      Fault

```